НАО «КАРАГАНДИНСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
ИМЕНИ АБЫЛКАСА САГИНОВА»

Кафедра кибербезопасность и искусственный интеллект

Лабораторная работа №1

по дисциплине «Безопасность баз данных, курсовой проект»

Тема: Управление доступом в СУБД SQL SERVER

Принял:
ст.преподаватель Жакина М.М.

_____
(подпись) *(дата)*

Выполнил:
ст.гр.СИБ-23-7 Гарифуллин

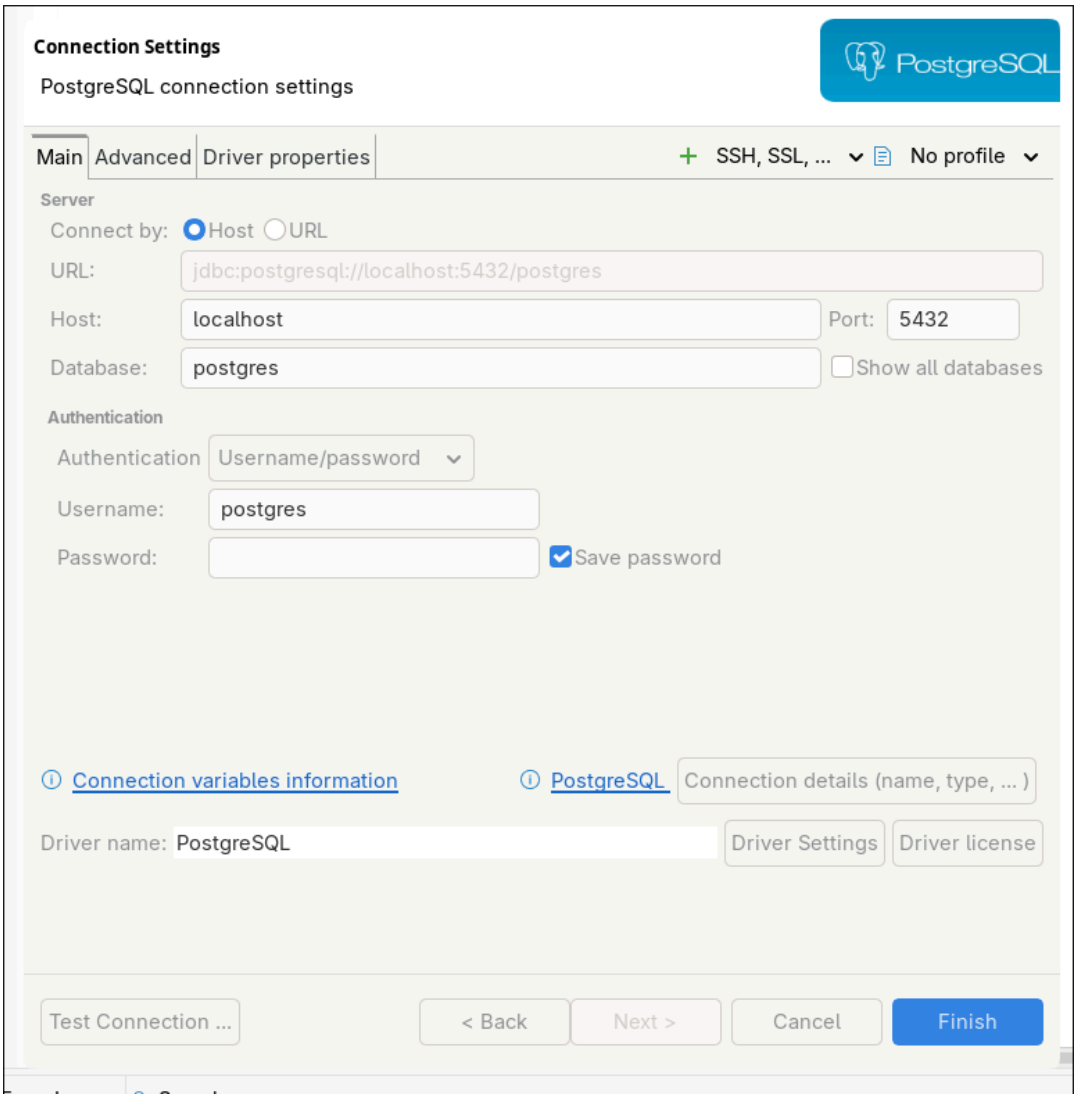
Караганда 2026

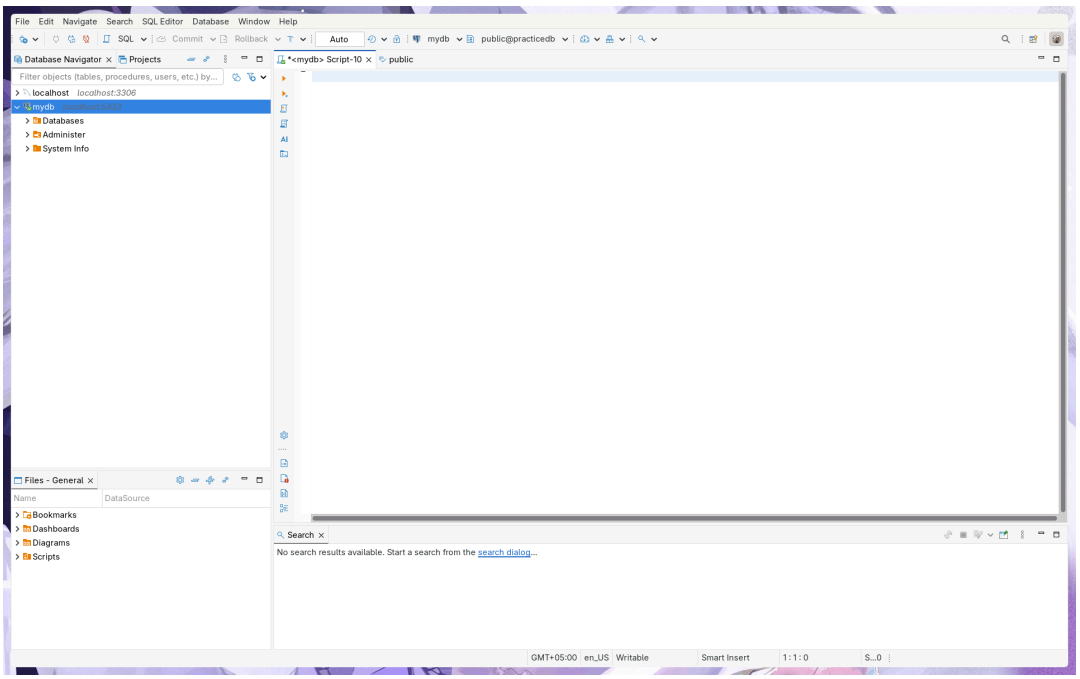
Цель практической работы: Изучение общих правил разграничения и предоставление прав доступа пользователям баз данных, архитектуры и

компонент системы безопасности SQL Server и режимов аутентификации пользователей, а также приобретение навыков администрирования системы безопасности: создания и управления учетными записями, управления пользователями, ролями и группами.

Задание 1. Подключитесь к серверу с помощью утилиты Management Studio.

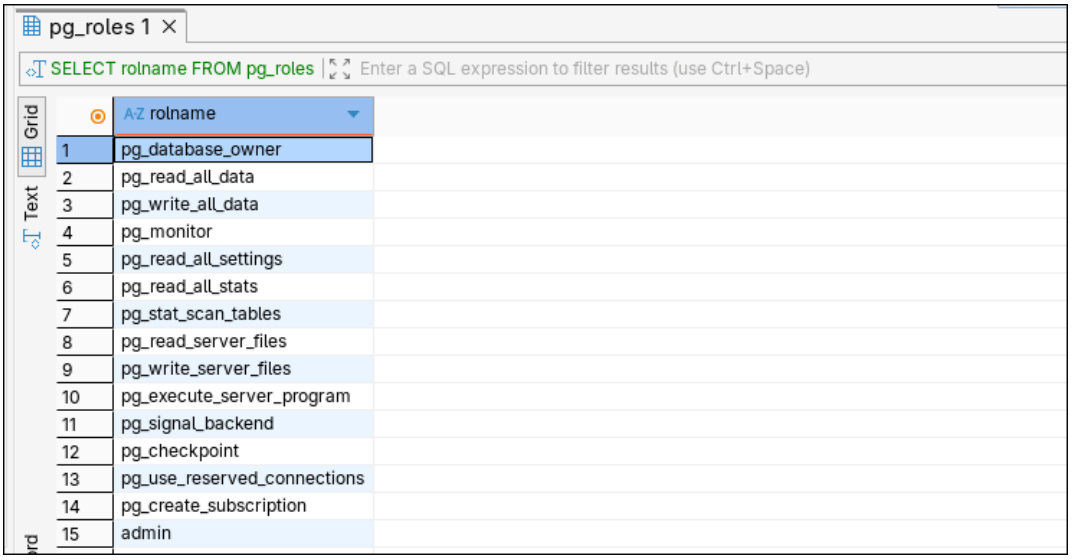
Окно подключения





Задание 2. Определите список ролей сервера

SELECT rolname FROM pg_roles;



Задание 3. Создайте и настройте новую учетную запись TempUser для входа в SQL Server.

Этап 1. Создание учетной записи.

В Microsoft SQL Server учетная запись для входа создается на уровне сервера. Для этого используется хранимая процедура:

sp_addlogin 'TempUser', 'Password!'

После выполнения этой команды создается login — объект сервера, который позволяет пользователю пройти аутентификацию и подключиться к СУБД. На этом этапе пользователь еще не имеет доступа к конкретной базе данных, создается только право входа на сервер.

В PostgreSQL аналог выполняется иначе. Отдельного объекта login не существует. Создается роль с правом LOGIN:

```
CREATE ROLE tempuser
 WITH LOGIN
 PASSWORD 'Password!';
```

Здесь роль сразу является учетной записью для входа.

Этап 2. Проверка создания учетной записи.

В MS SQL проверка выполняется с помощью процедуры:

sp_helplogins

Она выводит список логинов сервера, что подтверждает успешное создание TempUser.

В PostgreSQL проверка выполняется запросом к системному каталогу:

SELECT rolname FROM pg_roles;

Если роль tempuser присутствует в списке, учетная запись создана корректно.

pg_roles 1

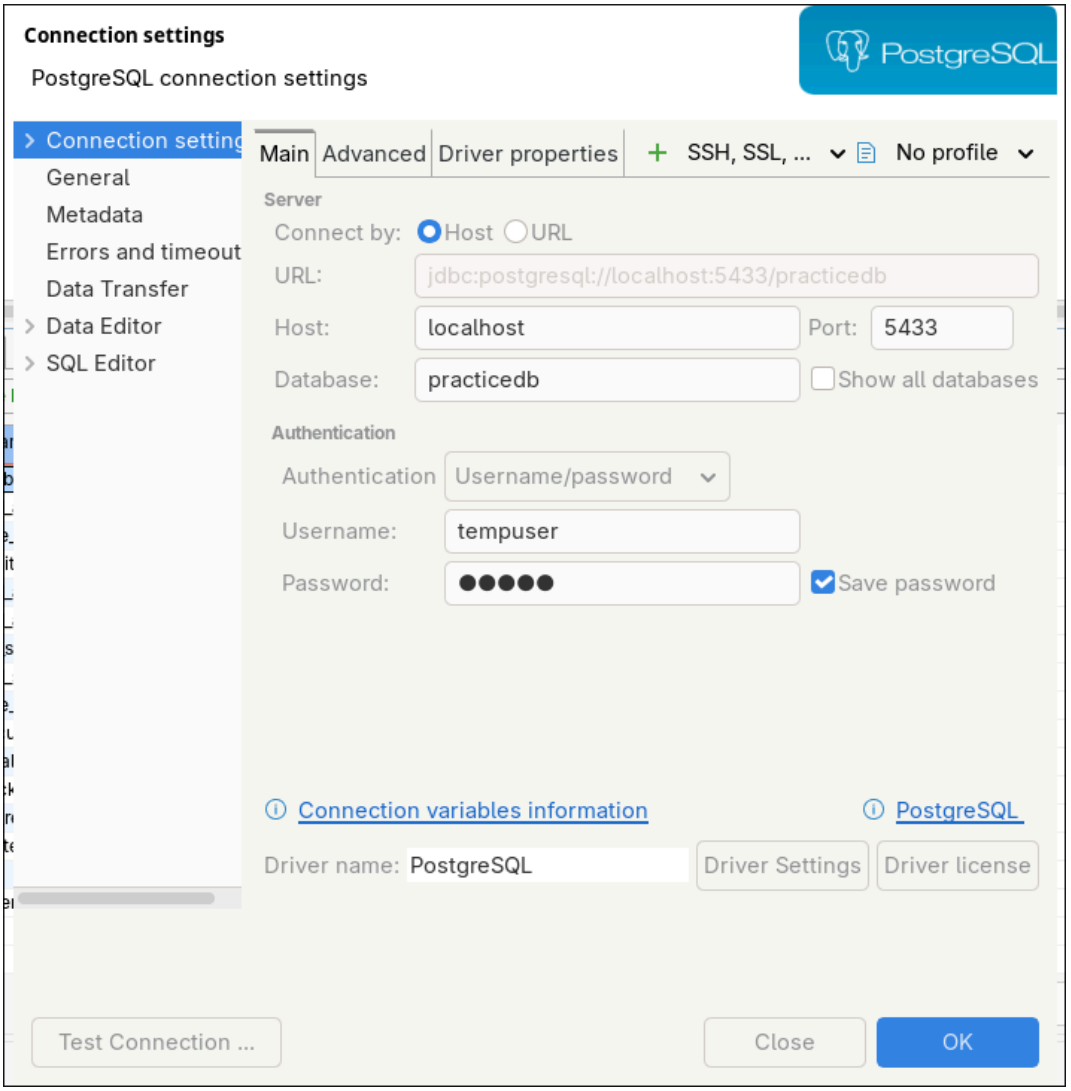
SELECT rolname FROM pg_roles | Enter a SQL expression to filter results (use Ctrl+Space)

	rolname
1	pg_database_owner
2	pg_read_all_data
3	pg_write_all_data
4	pg_monitor
5	pg_read_all_settings
6	pg_read_all_stats
7	pg_stat_scan_tables
8	pg_read_server_files
9	pg_write_server_files
10	pg_execute_server_program
11	pg_signal_backend
12	pg_checkpoint
13	pg_use_reserved_connections
14	pg_create_subscription
15	admin
16	tempuser

Этап 3. Проверка возможности входа.

В MS SQL выполняется выход из текущей сессии и попытка входа под TempUser через SQL Server Management Studio. Если подключение успешно, значит login работает.

В PostgreSQL выполняется создание нового подключения в DBeaver с использованием имени tempuser и заданного пароля. Если соединение устанавливается без ошибки аутентификации, учетная запись функционирует корректно.



Этап 4. Возврат под учетную запись администратора.

Этап 5. Назначение административной роли.

В MS SQL учетной записи TempUser назначается встроенная серверная роль:

sp_addsrvrolemember 'TempUser', 'securityadmin'

Роль securityadmin позволяет управлять логинами и их свойствами. Это серверный уровень безопасности.

В PostgreSQL встроенной роли securityadmin нет. Административные возможности задаются через атрибуты роли. Аналогично праву управления ролями используется:

ALTER ROLE tempuser WITH CREATEROLE;

Атрибут CREATEROLE позволяет создавать и изменять другие роли. Если требуется полный административный доступ, используется атрибут SUPERUSER, однако это более высокий уровень привилегий, чем securityadmin в MS SQL.

Таким образом, в MS SQL безопасность строится через разделение login и серверных ролей, а в PostgreSQL — через единый объект role с набором атрибутов и прав.

Задание 4. Определите список ролей базы данных *AdventureWorks2008* и членов роли *db_owner*.

Определение ролей базы данных и членов роли db_owner. В Microsoft SQL Server список ролей базы данных AdventureWorks2008 определяется с помощью:

sp_helprole

А члены роли db_owner определяются:

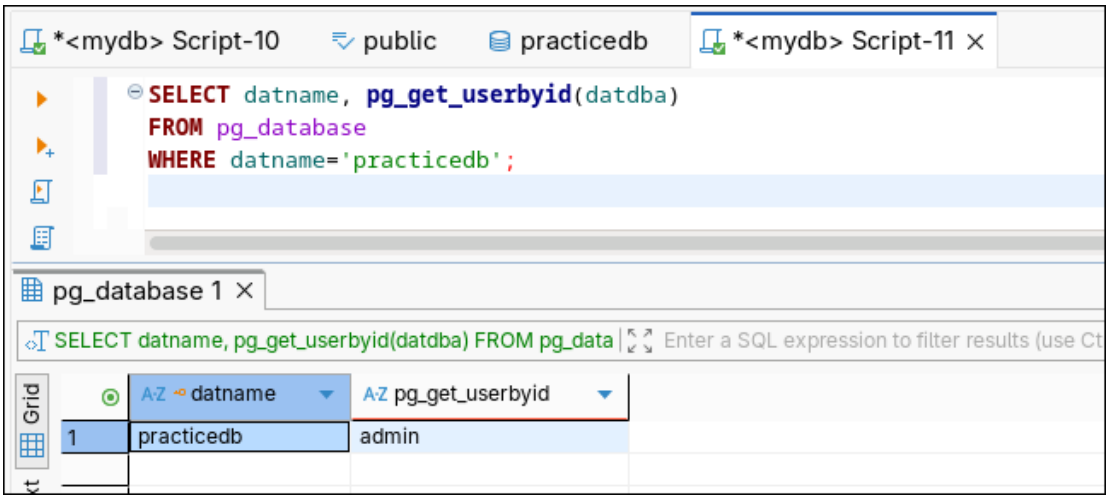
sp_helprolemember 'db_owner'

Роль db_owner предоставляет полный доступ ко всем объектам базы данных.

В PostgreSQL встроенной роли db_owner не существует. Полный доступ к базе данных имеет ее владелец (owner).
определил владельца базы данных (аналог db_owner) следующим образом:

```
SELECT datname, pg_get_userbyid(datdba)
FROM pg_database
WHERE datname='your_db';
```

Это подтверждает, какой пользователь имеет полный контроль над базой данных.



Задание 5. Создайте нового пользователя базы данных для логина TempUser.

Создание пользователя базы данных для логина TempUser. В MS SQL пользователь базы создается отдельно:

sp_adduser 'TempUser', 'MyFirstUser'

Проверка выполняется:

sp_helpuser

По умолчанию пользователю назначается роль public. Далее назначается роль db_datareader:

sp_addrolemember 'db_datareader', 'MyFirstUser'

Это дает права только на чтение данных.

В PostgreSQL отдельный пользователь базы не создается, так как роль tempuser уже существует.

Задание 6. Настройте права доступа пользователю *Andy*: предоставьте явным образом право только для выборки из таблицы *Product* и обновления только полей *Name* и *ListPrice* этой таблицы.

В PostgreSQL я создал пользователя:

CREATE ROLE andy
 WITH LOGIN
 PASSWORD '12345';

Предоставил право чтения таблицы:

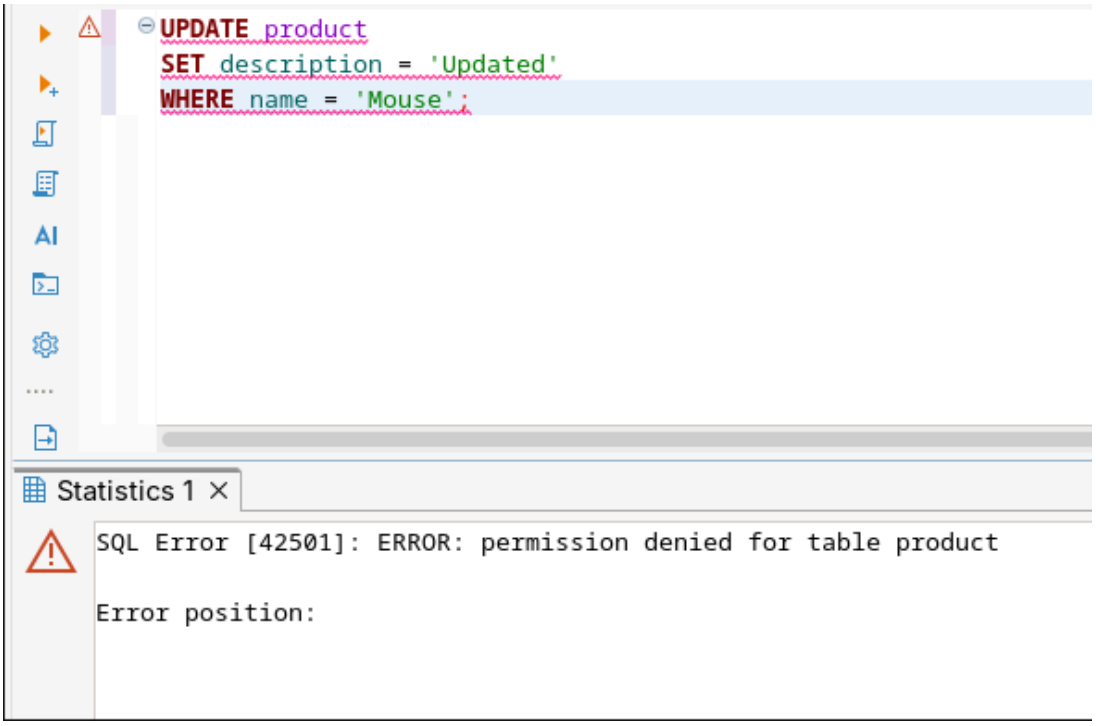
GRANT SELECT ON TABLE product TO andy;

И право изменения только определенных столбцов:

GRANT UPDATE (name, listprice)
 ON TABLE product
 TO andy;

Проверка запрещенного изменения:

UPDATE product
 SET description = 'Updated'
 WHERE name = 'Mouse';



Задание 7. Изучите выполнение вышеупомянутых функций при помощи графического интерфейса утилиты *Management Studio*.
В MSSMS большинство операций по управлению безопасностью выполняется через графический интерфейс.

Просмотр учетных записей для входа осуществляется через:

Security - Logins

В данном разделе отображаются все логины сервера и их параметры.
Создание новой учетной записи выполняется через:

Logins - New Login…

В появившемся окне на вкладке General указывается имя пользователя, тип аутентификации (SQL Server Authentication), пароль и база данных по умолчанию.
На вкладке Server Roles выбираются серверные роли (например securityadmin).
На вкладке User Mapping задается доступ к конкретной базе данных и назначаются роли базы данных.

Просмотр серверных ролей осуществляется через:

Security - Server Roles

В свойствах роли можно увидеть список пользователей, входящих в нее, а также разрешения. Встроенные роли изменить или удалить нельзя.

Управление пользователями базы данных выполняется через:

Database - Security - Users

Здесь отображается соответствие между Login Name (учетная запись сервера) и User Name (пользователь базы данных). Создание выполняется через New User…, где пользователь связывается с логином и включается в роли базы данных.

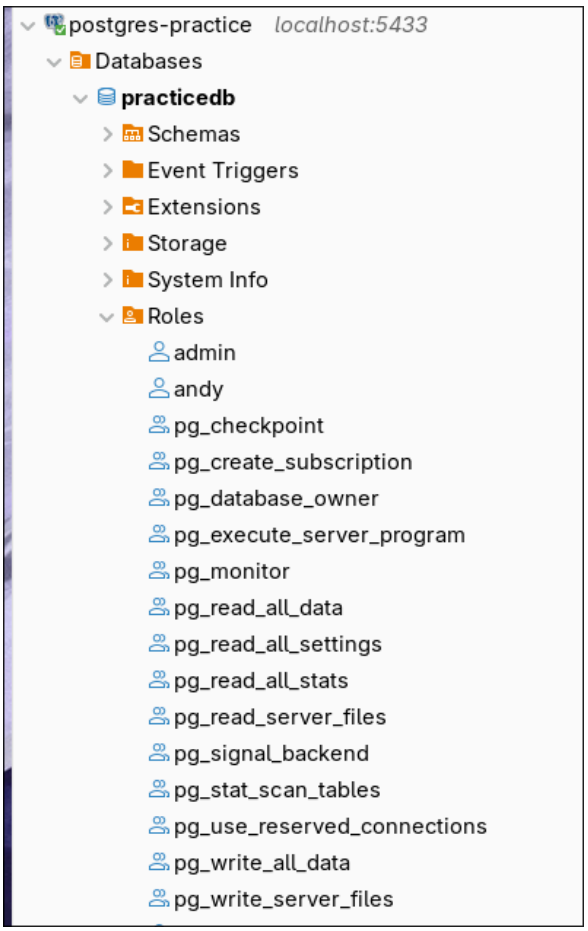
Просмотр ролей базы данных осуществляется через:

Database - Security - Roles

Назначение прав доступа выполняется через вкладку Securables, где можно явно разрешить (Grant), запретить (Deny) или оставить не назначенным (Revoke) доступ к объектам базы данных.

1. Просмотр созданных ролей выполнялся через:

Database - Roles



2. В PostgreSQL через DBeaver создание выполнялось через:

Roles - Create New Role

3. В MS SQL список серверных ролей просматривается через:

Security - Server Roles

В свойствах роли можно просмотреть список пользователей, входящих в нее.

В PostgreSQL встроенные серверные роли отсутствуют. Аналогичные административные возможности были назначены пользователю tempuser через свойства роли в DBeaver (атрибут CREATEROLE), что позволяет управлять другими учетными записями.

4. В MS SQL управление пользователями базы данных осуществляется через:

Database - Security - Users

Здесь создается пользователь базы данных и связывается с login.

В PostgreSQL отдельного пользователя базы данных нет. Роль используется напрямую. Доступ к базе данных был назначен через вкладку Privileges роли в DBeaver путем предоставления прав CONNECT и USAGE.

5. В MS SQL просмотр ролей базы данных осуществляется через:

Database - Security - Roles

В свойствах роли отображаются пользователи, входящие в нее.

6. В MS SQL назначение полномочий выполняется через вкладку:

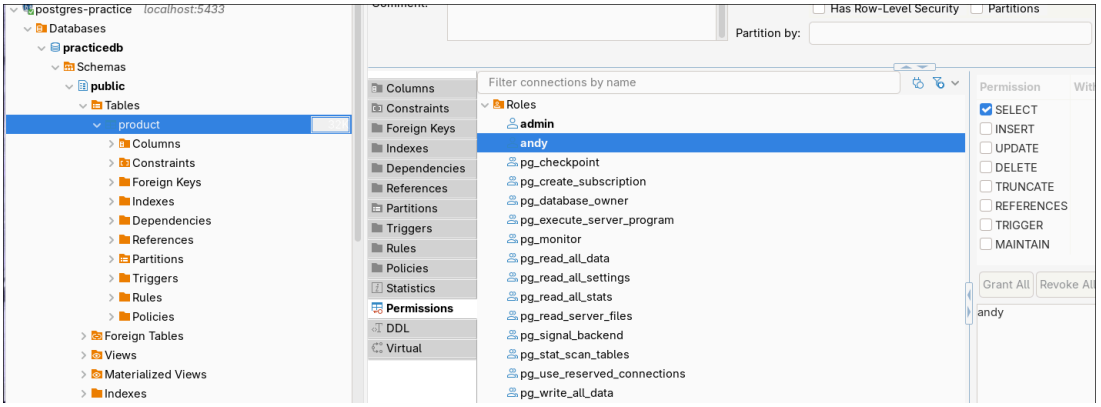
Securables

где можно явно разрешить или запретить доступ к объектам базы данных.

В PostgreSQL аналогичное назначение прав было выполнено в DBeaver через:

Schemas - public - Tables - product - Permissions

Пользователю andy были назначены:
 право SELECT на таблицу product
 право UPDATE только на столбцы name и listprice



Задание 8
 Отмена роли у учетной записи и удаление TempUser выполняется последовательно, так как удаление login невозможно, пока он связан с ролью и пользователем базы данных.

Сначала необходимо отменить включение пользователя в роль базы данных. Для этого в базе AdventureWorks2008 выполняется:

EXEC sp_droprolemember 'db_datareader', 'TempUser'

Данная команда удаляет пользователя TempUser из роли db_datareader.

После этого удаляется сам пользователь базы данных:

EXEC sp_dropuser 'TempUser'

На следующем этапе требуется отменить включение учетной записи TempUser в серверную роль. На сервере Microsoft SQL Server выполняется:

EXEC sp_dropsrvrolemember 'TempUser', 'securityadmin'

Это удаляет login TempUser из серверной роли securityadmin.

И только после выполнения предыдущих шагов можно удалить саму учетную запись для входа:

EXEC sp_droplogin 'TempUser'

В результате учетная запись TempUser полностью удаляется с сервера вместе со всеми ранее назначенными ей ролями.

В PostgreSQL нет разделения login и user. Есть одна сущность — role.
Поэтому процесс проще, но тоже требует последовательности.

Предположим, что TempUser имел права SELECT на таблицы и атрибут CREATEROLE.

Этап 1. Отмена административных прав (аналог удаления из securityadmin)

ALTER ROLE tempuser NOCREATEROLE;

Если бы были другие атрибуты (например SUPERUSER), они тоже снимаются через ALTER ROLE.

Этап 2. Отмена прав доступа к базе данных и объектам

REVOKE ALL PRIVILEGES ON DATABASE your_db FROM tempuser;

REVOKE ALL PRIVILEGES ON ALL TABLES IN SCHEMA public FROM tempuser;

REVOKE USAGE ON SCHEMA public FROM tempuser;

Это аналог удаления пользователя из ролей базы данных в MS SQL.

Этап 3. Удаление роли

DROP ROLE tempuser;

```
DROP ROLE tempuser;
```

Statistics 1

Name	Value
Updated Rows	0
Execute time	0.003s
Start time	Sat Feb 28 12:46:45 GMT+05:00 2026
Finish time	Sat Feb 28 12:46:45 GMT+05:00 2026
Query	DROP ROLE tempuser

Проверка, что учетная запись удалена:

SELECT rolname FROM pg_roles
 WHERE rolname = 'tempuser';

- Roles
  - admin
  - andy
  - pg_checkpoint
  - pg_create_subscription
  - pg_database_owner
  - pg_execute_server_program
  - pg_monitor
  - pg_read_all_data
  - pg_read_all_settings
  - pg_read_all_stats
  - pg_read_server_files
  - pg_signal_backend
  - pg_stat_scan_tables
  - pg_use_reserved_connections
  - pg_write_all_data
  - pg_write_server_files

Самостоятельная работа

1. Создайте 3 учетные записи для входа (логины) в MS SQL Server (SQL Server Authentication). Для каждого логина укажите подключение по умолчанию к вашей базе данных (БД по варианту).

```
CREATE ROLE viewer LOGIN PASSWORD 'viewer123';
CREATE ROLE editor LOGIN PASSWORD 'editor123';
CREATE ROLE limited LOGIN PASSWORD 'limited123';
```

2. В вашей базе данных (БД по варианту) создайте 3 пользователя на основе созданных логинов.

В Postgresql нету логинов

3. Назначьте каждому пользователю роль(и) одним из способов (на выбор):

− использовать встроенные роли базы данных (например: db_datareader, db_datawriter, db_ddladmin и др.);

− создать собственные роли и назначить им разрешения, соответствующие структуре вашей базы данных.

Создание ролей:

```
CREATE ROLE db_viewer;
 CREATE ROLE db_editor;
 CREATE ROLE db_limited;
```

Назначение ролей:

```
GRANT db_viewer TO viewer;
 GRANT db_editor TO editor;
 GRANT db_limited TO limited;
```

4. Для ролей определите разрешения доступа (по вашему выбору), но в итоге должны быть реализованы минимум три разных уровня доступа для трех пользователей, например:

− пользователь с правами только просмотра данных (SELECT);

− пользователь с правами изменения данных (INSERT/UPDATE/DELETE) для выбранных таблиц;

− пользователь с ограниченным доступом (например, запрет на одну из таблиц или доступ только к определенным объектам).

Только просмотр данных:

GRANT SELECT ON students TO db_viewer;
GRANT SELECT ON grades TO db_viewer;

Изменение данных:

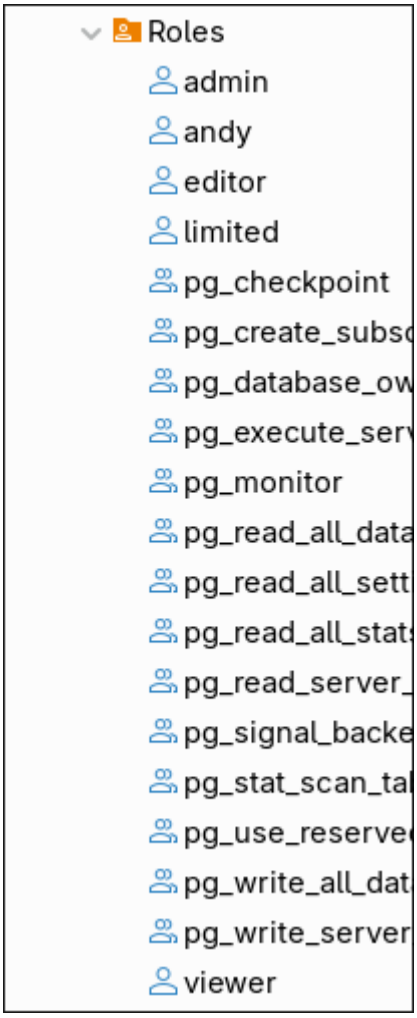
GRANT INSERT, UPDATE, DELETE ON students TO db_editor;
GRANT INSERT, UPDATE, DELETE ON grades TO db_editor;

Ограниченный доступ:

GRANT SELECT ON students TO db_limited;
REVOKE ALL ON grades FROM db_limited;

5. Докажите правильность выполненных действий:

- покажите, что логины, пользователи и роли созданы;



- подтвердите назначение ролей пользователям;

Statistics 1

```
SQL Error [42501]: ERROR: permission denied for table students
```

```sql
SET ROLE viewer;
SELECT * FROM students;
INSERT INTO students VALUES (3,'Test','IT-99');

RESET ROLE
```

```sql
SET ROLE editor;
INSERT INTO students(fullname,groupname)
VALUES ('New Student','IT-30');

RESET ROLE;
```

Statistics 1

Name	Value
Updated Rows	1
Execute time	0.012s
Start time	Sat Feb 28 13:03:30 GMT+05:00 2026
Finish time	Sat Feb 28 13:03:30 GMT+05:00 2026
Query	INSERT INTO students(fullname,groupname) VALUES ('New Student','IT-30')

```sql
SET ROLE limited;
SELECT * FROM students;
SELECT * FROM grades;

RESET ROLE;
```

grades 1

SET ROLE limited; SELECT * FROM students; SELECT * FI | Enter a SQL expression to filter results (

```
SQL Error [42501]: ERROR: permission denied for table students

Error position:
```